

# ENI-ABT-2018

## *bases de données*

### SQL

#### Module 2

Chargé de cours  
Dr Yacouba Goita

# Objectifs d'apprentissage

- Décrire la syntaxe du SQL.
- Formuler des requêtes simples et complexes en SQL
- Implémenter les contraintes d'intégrités en SQL
- Implémenter une base de donnée et pouvoir l'interroger

# Plan

- Introduction
- DDL: Data Definition Language
- DML: Data Manipulation Language
- DCL : Data Control Language

# Introduction

- L'algèbre relationnelle permettait de définir théoriquement les requêtes.
- Pour exprimer les requêtes théoriques dans un SGBD, on utilise une forme de langage de requête structuré appelée Structured Query Language ou SQL.
- SQL est le plus répandu des langages de requête de bases de données en usage aujourd'hui.

# Introduction

- C'est un langage complet de systèmes de gestion de base de données.
- Il permet de manipuler les composantes d'une base de données, mais aussi de les créer et d'en assurer la sécurité.

# Trois langages

- SQL est composé des 3 langages suivants:
  - DDL (*Data definition language*): Composant autorisant la définition (création) de composantes de base de données. Par exemple: les tables.
  - DML (*Data manipulation language*): Composant permettant le traitement de l'information de la bases de données. Par exemple: extraction, ajout et modification de données.

# Trois langages

- DCL (*Data Control Language*): Composant fournissant la sécurité interne de la base de données. Par exemple: usagers, mot de passe, permissions.
- Nous allons nous concentrer sur les 2 premiers langages.

# SQL

- SQL a été développé par IBM à San Jose, Californie. La version courante de SQL est appelée SQL-92 (due à l'année de normalisation).
- Toutefois, les produits commerciaux qui supportent SQL utilisent en partie SQL-92 et offrent même des fonctionnalités de plus.
- SQL est un langage non-procédural, ce qui signifie que les expressions SQL stipulent ce qui doit être fait et non comment cela doit être fait.



# SQL

## ● Notes:

- On peut insérer des sauts de lignes dans une instruction SQL à n'importe quel endroit, ce qui est pratique pour augmenter la lisibilité.

# Plan

- Introduction
- DDL: Data Definition Language
- DML: Data Manipulation Language

# DDL

- Les instructions pour la définition des données sont:

- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- CREATE INDEX

- et aussi (à voir plus tard)

- CREATE TRIGGER
- ...

# Create Table

- Soit la relation EDITEURS (PubID\*, PubNom, PubPhone)
- Toute relation du modèle relationnel devient une table dans la BD.
- Pour créer une table, on utilise l'instruction CREATE TABLE:

# Create Table

- Création simple d'une table:

```
CREATE TABLE nom_table  
(nom_col1 TYPE1,  
nom_col2 TYPE2,  
...)
```

- Création (simple) du schéma de table EDITEURS (on définirait la clé ultérieurement)

```
CREATE TABLE EDITEURS  
(PubID TEXT(10),  
PubNom TEXT(100),  
PubPhone TEXT(20))
```

# Contraintes de Create Table

- Plusieurs contraintes peuvent être définies sur les champs de la table.
- Les contraintes sont:
  - PRIMARY KEY: Désigner une clé primaire
  - REFERENCES: Désigner une clé étrangère, établissant une relation entre 2 tables
  - UNIQUE: Chaque ligne de la table doit avoir une valeur différente ou NULL pour cette (ou ces) colonne(s).
  - NOT NULL: Interdire les valeurs NULL
  - CHECK: Restreindre les valeurs autorisées à une plage spécifiée.

# Contraintes de Create Table

- Il y a 2 types de clauses de contraintes dans une instruction CREATE TABLE:
  - Contrainte de colonne unique
  - Contrainte de colonne multiple

# Contraintes de colonne unique

- Contrainte de colonne unique
  - Elle est utilisée à l'intérieur d'une définition de colonne.
  - Elle sert à définir une contrainte qui ne s'applique qu'à une seule colonne, celle incluse dans la définition.

PubNom TEXT(100) NOT NULL

ou bien

PubNom TEXT(100) Constraint valeurNulle NOT NULL,



# Contraintes de colonne unique

- Nous définirons la clé primaire de EDITEURS et spécifier que PubNom ne peut avoir une valeur nulle.
- Création du schéma de table EDITEURS  
CREATE TABLE EDITEURS  
(PubID TEXT(10) Constraint cléPrimaire Primary Key,  
PubNom TEXT(100) Not Null,  
PubPhone TEXT(20))

# Contraintes de colonne unique

- Création du schéma de table LIVRES et du lien à EDITEURS utilisant PubID comme clé étrangère :

```
CREATE TABLE LIVRES  
(ISBN TEXT(13) CONSTRAINT ClePrimaire PRIMARY  
  KEY,  
  Titre TEXT(100) NOT NULL,  
  Prix MONEY CONSTRAINT contPrix CHECK (Prix>19),  
  PubID TEXT(10),CONSTRAINT cleEtrangere FOREIGN  
  KEY (PubID) REFERENCES EDITEURS (PubID));
```

# Contraintes multi-colonnes

- Contrainte multi-colonne
  - Les contraintes multi-colonne servent lorsque plusieurs colonnes sont en jeu dans la contrainte. Clé primaire double, clé étrangère double, etc.

CONSTRAINT Nom

[CHECK(expressionDeContrainte)| PRIMARY KEY  
(NomColonne,...) |

UNIQUE(NomColonne,...) |

FOREIGN KEY (ColonneReference,...)

REFERENCES

TableReference[(ColonneReference,...)]

# Contraintes multi-colonnes

## ● exemple:

- Création du schéma de table Personne où chaque personne est identifiée par son nom et prénom (par exemple)

```
CREATE TABLE Personne  
(Nom TEXT(10)  
  Prenom TEXT(100),  
  Adresse TEXT(20)  
  CONSTRAINT ClePrimaire PRIMARY KEY (Nom,  
    Prenom));
```

# Contraintes

- Les paramètres de l'instruction CREATE TABLE sont un nom de table suivi par une ou plusieurs définitions de colonnes, suivies par 0, une ou plusieurs contraintes multi-colonnes.

- Définition de colonne

DefinitionColonne ::= NomColonne

TypeDonnee[(taille)]

[ContrainteColonneSimple]

# Contraintes

- Note:

- Si on spécifie une clé étrangère, l'intégrité référentielle est garanti, i.e. le dossier de la table parent ne peut être effacé s'il existe des enregistrements correspondant dans la table enfant.
- Note:parent=contient la clé primaire, enfant=contient la clé étrangère correspondante.

# Contraintes

- Note:

- Pour garantir l'intégrité référentielle, il faut spécifier ce qu'il faudrait faire en modifiant ou supprimant une table parente.
- SQL permet de spécifier ON UPDATE CASCADE, ON DELETE CASCADE ou ON DELETE set NULL.
- `CONSTRAINT NomIndex FOREIGN KEY (PubID) REFERENCES EDITEURS (PubID) ON DELETE CASCADE`
- Ainsi, si on efface le dossier correspondant de la table Editeurs tous les dossiers de la table courante seront effacés.

# Contraintes

- CONSTRAINT NomIndex FOREIGN KEY (PubID) REFERENCES EDITEURS (PubID) ON UPDATE CASCADE
- Ainsi, si on met à jour la clé du dossier correspondant de la table Editeurs tous les dossiers de la table courante seront mis à jour.
- On peut aussi utiliser le SET NULL à la place de CASCADE: CONSTRAINT NomIndex FOREIGN KEY (PubID) REFERENCES EDITEURS (PubID) ON DELETE SET NULL
- Ainsi, si l'enregistrement de la table parent est effacé, les enfants correspondants ne seront pas effacés et leur clé étrangère sera mise à NULL.



# Alter Table

- ALTER TABLE
  - Avec cette commande on peut:
    - Ajouter une nouvelle colonne
    - Supprimer une colonne

Par exemple:

```
ALTER TABLE Livre ADD COLUMN prixCoutant CURRENCY;
```

# Alter Table

- ALTER TABLE
  - La syntaxe

ALTER TABLE

NomTable

ADD COLUMN NomColonne Type[(grandeur)][contrainte]

DROP COLUMN NomColonne |

ADD CONSTRAINT ContrainteMultiColonne

DROP CONSTRAINT ContrainteMultiColonne

# Drop

## ● DROP

- Permet d'effacer une table ou un index ou même une BD

ex: DROP TABLE Livre;

- La syntaxe

DROP TABLE NomTable | DROP INDEX NomIndex  
ON NomTable

# Plan

- Introduction
- DDL: Data Definition Language
- DML: Data Manipulation Language

# SQL DML

- Langage de manipulation de données (DML)
  - SELECT
  - UNION
  - UPDATE
  - DELETE
  - INSERT INTO
  - SELECT INTO

# Select

- L'instruction SELECT

- Permet de lire les informations contenues dans la base de données.

- Par exemple:

```
SELECT * FROM livre WHERE prix > 10.
```

Cette instruction permet de fournir tous les tuples de la table livre dont le prix est supérieur à 10.

# Select

- L'instruction SELECT
  - Sa syntaxe est :  
SELECT [prédicat] Colonne,...  
FROM ExpressionTable  
[WHERE Condition]  
[GROUP BY CritèreGroupe]  
[HAVING Condition]  
[ORDER BY CritèreOrdre]

# Select

- Prédicat
  - Sera vu un peu plus loin
- Colonne
  - La colonne ou combinaisons de colonnes à retourner.
  - La description peut prendre l'une des formes suivantes:
    - \* (l'astérisque désigne toutes les colonnes)
    - Le nom d'une colonne
    - Une expression combinant des noms de colonnes et des opérateurs;
    - On peut répéter Colonne autant de fois que l'on veut.



# Select

- Lorsque 2 colonnes retournées (de différentes tables) ont le même nom, il est nécessaire d'ajouter le nom de la table devant le nom de la colonne séparé par un point;
- Par exemple:

On ne peut écrire:

```
SELECT titre, pubID, pubID FROM livre, Editeur
```

Ça porte à confusion. On ne sait plus à quel pubID se référer.

# Select

- On doit plutôt écrire:

`SELECT titre, livre.pubID, editeur.pubID FROM livre, Editeur`

- Finalement, chaque Colonne peut se terminer par AS NomAlias pour donner un nouveau nom. C'est le renommage de colonnes.

- Par exemple

`SELECT titre, livre.pubID AS LID, editeur.pubID AS EID FROM  
livre, Editeur`

# Select

- La clause FROM ExpressionTable
  - Cette clause spécifie les tables (ou requêtes) dont l'instruction SELECT extrait ses lignes.
  - ExpressionTable peut être un nom de table unique, plusieurs nom de tables séparés par des virgules, ou une clause de jointure. Elle peut aussi avoir un AS NomAlias.

# Select

- Par exemple:

```
SELECT * FROM Auteur;  
SELECT * FROM Auteur,Editeur;  
SELECT * FROM ClauseJointure;  
SELECT * FROM Auteur AS A, Editeur AS B;
```

- La projection de l'algèbre relationnelle,  $\Pi$ , est exprimée sous la forme

```
SELECT [prédicat] Colonne,...  
FROM ExpressionTable
```

# Select

- La clause WHERE Condition
  - En algèbre: Restriction  $\sigma$
  - WHERE permet de spécifier des conditions quant aux lignes retournées.
  - Les expressions peuvent comporter des noms de colonne, constantes, comparaisons ( $=$ ,  $<$ ,  $>$ ,  $<=$ ,  $>=$ ,  $<>$ , BETWEEN), des relations (AND, OR, XOR, NOT) et des fonctions.

# SQL/alg.

- $\Pi_{\text{solde, nom}}$  (compte) devient  
Select solde, nom from compte.
- $\sigma_{\text{solde} > 2200}$  (compte) devient  
Select \* from compte where solde > 2200
- $(\Pi_{\text{nom}} (\sigma_{\text{solde} > 2200} (\text{compte})))$  devient  
Select nom from compte where solde > 2200
- $\rho_{\text{client}}(\text{numclient}, \text{nomclient}, \text{soldeclient})$  (compte (no-compte, nom, solde))  
devient  
Select no-compte AS numclient, nom AS nomclient, solde AS soldeclient  
From compte AS client.

# SQL jointure

## ● Clause de jointure

- Notons qu'une clause de jointure n'est pas une instruction SQL par elle-même, mais doit être placée à l'intérieur d'une instruction SQL.
- Les jointures permettent de traiter les données de 2 ou plusieurs tables dans la même requête.
- Produit cartésien
  - Le résultat renvoie toutes les lignes possibles des tables sélectionnées (livre x Editeur)  
SELECT titre, pubID, pubID FROM livre, Editeur

# SQL jointure

- Jointure interne(naturelle et Thêta)
  - La syntaxe d'une clause de jointure interne est la suivante:  
Select colonne,....  
From Table1, Table2  
WHERE Table1.Colonne1 = Table2.Colonne1  
[{AND|OR condition jointure thêta...}]
  - Ce type de jointure permet de trouver exclusivement les dossiers des 2 tables qui correspondent par les colonnes mentionnées.



# SQL/alg. jointure

●  $\text{client} \bowtie \text{compte} = (\sigma_{\text{Client.nom} = \text{Compte.nom}}(\text{client} \times \text{compte}))$

Select \*

From client, compte

Where client.nom = compte.nom

●  $\sigma_{\text{Compte.solde} > 1800}(\text{client} \bowtie \text{compte})$ .

Select \*

From client, compte

Where (client.nom = compte.nom) and (compte.solde > 1800)

●  $\Pi_{\text{client.rue}, \text{compte.solde}}(\text{client} \bowtie_{\theta} \text{compte})$  avec  $\theta = (\text{Compte.solde} > 1800)$

Select client.rue, compte.solde

From client, compte

Where (client.nom = compte.nom) and (compte.solde > 1800)

# SQL jointure

## ● Jointure externe

- Pour exprimer la jointure externe, il faut placer un (+) au niveau du champs où la jointure externe serait appliquée.
- En SQL, on ne peut exprimer la jointure externe. On exprime la jointure externe gauche ou droite.
- Le (+) se place sur le champ de droite pour une jointure externe gauche et à gauche pour une jointure externe droite.
- ```
select A.col1, A.col2, B.col1, B.col2  
from A, B  
where A.col1 = B.col1(+);
```

# SQL jointure

- Editeur  $\rightarrow$  Livre

Select \*

From Editeur, Livre

Where Editeur.NoE = Livre.NoE (+)

- $\Pi_{\text{nom, no-dep}}(\text{employe} \rightarrow \text{departement})$

Select employe.nom, departement.no-dep

From employe, departement

Where employe.nas = departement.nas-g (+)

- Editeur  $\leftarrow$  Livre

Select \*

From Editeur, Livre

Where Editeur.NoE (+) = Livre.NoE

# SQL

- Langage de manipulation de données (DML) (suite)
  - SELECT (suite)
  - UNION
  - INTERSECT
  - EXCEPT (MINUS)
  - UPDATE
  - DELETE
  - INSERT INTO
  - SELECT INTO
  - Les sous-requêtes
  - VIEW

# SELECT

## ● Rappel SELECT

```
SELECT [prédicat]DescriptionColonne,...  
FROM ExpressionTable  
[WHERE Condition]  
[GROUP BY CritèreGroupe]  
[HAVING Condition]  
[ORDER BY CritèreOrdre]
```

# SELECT (prédicat)

## ● SELECT

SELECT [prédicat]DescriptionColonne,...

- Il nous permet de faire des choix quant aux lignes identiques qui seraient retournées par la requête. On peut utiliser les prédicats suivants: ALL, DISTINCT.
  - ALL (par défaut) retourne toutes les lignes lues sans exception.
  - DISTINCT ne retourne que les lignes lues qui sont différentes. S'il y a des lignes pareilles, seule la première apparaîtra.

# SELECT (prédicat)

- Par exemple:

```
SELECT EDITEURS.PubNom  
FROM EDITEURS, LIVRES WHERE  
    EDITEURS.PubID = LIVRES.PubID;
```

- ici, puisque qu'un éditeurs a plusieurs livres, son nom apparaîtra plusieurs fois.

```
SELECT DISTINCT PubNom  
FROM EDITEURS, LIVRES WHERE  
    EDITEURS.PubID = LIVRES.PubID;
```

- Ici, même si l'éditeurs a plusieurs livres, son nom n'apparaîtra qu'une fois.

# EXEMPLE

Sans distinct

Table Editeurs

| PubID | PubNom       | PubPhone     | Pays            |
|-------|--------------|--------------|-----------------|
| 1     | Big House    | 123-456-7890 | Etats-Unis      |
| 2     | Medium House | 999-999-9999 | Grande-Bretagne |
| 3     | Small House  | 714-000-0000 | France          |
| 4     | Alpha Press  | 814-555-1111 | Allemagne       |

| PubNom       |
|--------------|
| Big House    |
| Big House    |
| Big House    |
| Big House    |
| Big House    |
| Big House    |
| Medium House |
| Medium House |
| Medium House |
| Small House  |
| Small House  |
| Small House  |
| Small House  |
| Small House  |
| Small House  |

Avec distinct

| PubNom       |
|--------------|
| Big House    |
| Medium House |
| Small House  |



# SELECT (GROUP BY)

| Company   | Amount |
|-----------|--------|
| W3Schools | 5500   |
| IBM       | 4500   |
| W3Schools | 7100   |

« SELECT Compagnie, SUM(Montant) FROM Ventes » est refusée alors que:  
“SELECT Company,SUM(Amount) FROM Sales GROUP BY Company”  
donne...

| Company   | SUM(Amount) |
|-----------|-------------|
| W3Schools | 12600       |
| IBM       | 4500        |

# SELECT (HAVING)

- Clause HAVING CritereGroupe
  - La clause HAVING est le filtrage des groupes de l'algèbre.
  - La clause HAVING a été introduite car le WHERE ne peut pas être utilisé avec les fonctions d'agrégation comme SUM.
  - Sans HAVING, il ne serait pas possible de tester les conditions portant sur le résultat d'une fonction

# SELECT (HAVING)

- La clause HAVING n'affecte que les valeurs qui sont affichées
- La clause HAVING ne s'applique qu'après le regroupement.
- Le critère du WHERE s'applique sur un champ (et non sur une agrégation)
- Le critère HAVING s'applique sur un champ regroupé (inclus dans la clause GROUP BY) ou un champ agrégé (inclus dans une fonction d'agrégation).

# EXEMPLE

- Par exemple:

```
SELECT Editeurs.Pubnom, AVG(Prix) AS PrixMoyen  
FROM Editeurs, Livres WHERE Editeurs.PubID =  
    Livres.PubID
```

```
GROUP BY Editeurs.PubNom HAVING AVG(Prix) < 25.00;
```

- Ici, la requête demande le nom des éditeurs et le prix minimum des livres de ces éditeurs dont le prix moyen est inférieur à 25.00\$.

# EXEMPLE

```
SELECT Editeurs.Pubnom, AVG(Prix) AS PrixMoyen  
FROM Editeurs, Livres WHERE Editeurs.PubID =  
    Livres.PubID AND Prix < 25.00 GROUP BY  
    Editeurs.PubNom;
```

- Ici, la requête demande le nom des éditeurs et le prix minimum des livres de ces éditeurs dont le prix est inférieur à 25.00\$.
- Notez qu'on ne peut utiliser l'alias PrixMoyen dans le HAVING

# EXEMPLE

La requête sans  
critère

| Pubnom       | PrixMoyen  |
|--------------|------------|
| Big House    | 23,3333333 |
| Medium House | 31,6666667 |
| Small House  | 34,5       |

Premier exemple  
avec le HAVING

| Pubnom    | PrixMoyen  |
|-----------|------------|
| Big House | 23,3333333 |

Deuxième exemple  
sans le HAVING et  
avec le WHERE

| Pubnom       | PrixMoyen |
|--------------|-----------|
| Big House    | 17,5      |
| Medium House | 12        |
| Small House  | 20        |

# Exemple

| <b>Company</b> | <b>Amount</b> |
|----------------|---------------|
| W3Schools      | 5500          |
| IBM            | 4500          |
| W3Schools      | 7100          |

SELECT Company, SUM(Amount) FROM Sales GROUP BY Company HAVING SUM(Amount)>10000 donne...

| <b>Company</b> | <b>SUM(Amount)</b> |
|----------------|--------------------|
| W3Schools      | 12600              |

# SELECT (ORDER BY)

| Company   | OrderNumber |
|-----------|-------------|
| Sega      | 3412        |
| ABC Shop  | 5678        |
| W3Schools | 2312        |
| W3Schools | 6798        |

SELECT Company, OrderNumber FROM Orders ORDER BY Company (ordre alphabétique)

| Company   | OrderNumber |
|-----------|-------------|
| ABC Shop  | 5678        |
| Sega      | 3412        |
| W3Schools | 6798        |
| W3Schools | 2312        |



# SELECT (ORDER BY)

SELECT Company, OrderNumber FROM Orders ORDER BY Company DESC, OrderNumber ASC (trie les compagnies par ordre alphabétique décroissant et les OrderNumber par ordre numérique croissant)

| Company   | OrderNumber |
|-----------|-------------|
| W3Schools | 2312        |
| W3Schools | 6798        |
| Seqa      | 3412        |
| ABC Shop  | 5678        |

# UNION

## ● L'Opération UNION

- L'opération UNION est utilisée pour créer l'union de 2 ou plusieurs tables. La syntaxe est:  
Requete {UNION [ALL] Requete},...
- Où Requete est soit une instruction SELECT ou le nom d'une requête stockée (sera vue au prochain cours).
- Par défaut, les doublons sont triés entre les deux requêtes et n'apparaissent qu'une seule fois.
- L'option ALL force à inclure tous les enregistrements et donc de voir les doublons.
- ALL améliore les performances car il n'y a pas de tri de doublon.

# UNION

## ● Par exemple

```
SELECT * FROM Livres
```

```
UNION ALL
```

```
SELECT * FROM NewLivres WHERE Prix > 25.00
```

```
ORDER BY Titre;
```

## ● Notes

- Toutes les requêtes dans une opération UNION doivent retourner le même nombre de champs.
- Les colonnes sont combinées dans l'union selon leur ordre dans la clause de requêtes et non par leur nom

# EXEMPLE

Résultats de l'union

Table Livres

| ISBN          | Titre           | PubID | Prix  |
|---------------|-----------------|-------|-------|
| 0-103-45678-9 | Iliad           | 1     | 25,00 |
| 0-11-345678-9 | Moby Dick       | 3     | 49,00 |
| 0-12-333433-3 | On Liberty      | 1     | 25,00 |
| 0-123-45678-0 | Ulysses         | 2     | 34,00 |
| 0-12-345678-9 | Jane Eyre       | 3     | 49,00 |
| 0-321-32132-1 | Balloon         | 3     | 34,00 |
| 0-55-123456-9 | Main Street     | 3     | 25,00 |
| 0-555-55555-9 | MacBeth         | 2     | 12,00 |
| 0-91-045678-5 | Hamlet          | 3     | 20,00 |
| 0-91-335678-7 | Fairie Queene   | 1     | 15,00 |
| 0-99-777777-7 | King Lear       | 2     | 49,00 |
| 0-99-999999-9 | Emma            | 1     | 20,00 |
| 1-1111-1111-1 | C++             | 1     | 30,00 |
| 1-22-233700-0 | Visual Basic    | 1     | 25,00 |
| 1-56592-297-2 | Access Database | 3     | 30,00 |

| ISBN          | Titre           | PubID | Prix |
|---------------|-----------------|-------|------|
| 1-56592-297-2 | Access Database | 3     | 30   |
| 0-321-32132-1 | Balloon         | 3     | 34   |
| 1-1111-1111-1 | C++             | 1     | 30   |
| 0-99-999999-9 | Emma            | 1     | 20   |
| 0-91-335678-7 | Fairie Queene   | 1     | 15   |
| 0-91-045678-5 | Hamlet          | 3     | 20   |
| 0-103-45678-9 | Iliad           | 1     | 25   |
| 0-12-345678-9 | Jane Eyre       | 3     | 49   |
| 0-12-335671-9 | John Bull       | 3     | 39   |
| 0-99-777777-7 | King Lear       | 2     | 49   |
| 0-555-55555-9 | MacBeth         | 2     | 12   |
| 0-55-123456-9 | Main Street     | 3     | 25   |
| 0-11-345678-9 | Moby Dick       | 3     | 49   |
| 0-12-333433-3 | On Liberty      | 1     | 25   |
| 0-11-354678-9 | Paris           | 3     | 29   |
| 0-123-45678-0 | Ulysses         | 2     | 34   |
| 1-22-233700-0 | Visual Basic    | 1     | 25   |

| ISBN          | Titre     | PubID | Prix  |
|---------------|-----------|-------|-------|
| 0-103-45258-9 | Corba     | 1     | 25,00 |
| 0-11-354678-9 | Paris     | 3     | 29,00 |
| 0-12-333883-3 | SQL       | 1     | 15,00 |
| 0-12-335671-9 | John Bull | 3     | 39,00 |

Table  
newLivres

# UNION

- AS peut être utilisé dans la première instruction SELECT pour modifier les noms de colonnes retournées.
- Dans un UNION, on ne peut appliquer un ORDER BY pour chaque requête. Le ORDER BY doit être appliqué aux deux requêtes en même temps.
- La clause ORDER BY est utilisée à la fin de la dernière requête pour ordonner les données retournées des deux requêtes. Elle utilise toutefois les noms de colonnes de la première requête.
- GROUP BY et HAVING peuvent être utilisés dans chaque partie.
- UNION ne fait pas partie de SQL-92 mais est aussi présent dans Oracle et d'autres SGBD.

# Intersect

- Syntaxe

- SELECT ...  
**INTERSECT**  
SELECT ...

- Exemple:

```
SELECT t1.col1, t1.col2 FROM t1  
INTERSECT
```

```
SELECT t2.col1, t2.col2 FROM t2
```

# EXCEPT (MINUS)

- Pour exprimer la difference en SQL standard:

- SELECT ...  
**EXCEPT**  
SELECT ...

- Exemple:

```
SELECT t1.col1, t1.col2 FROM t1
```

```
EXCEPT
```

```
SELECT t2.col1, t2.col2 FROM t2
```

En oracle, on utilise le mot **MINUS**

# UPDATE

- L'instruction UPDATE

- L'instruction UPDATE est utilisée pour mettre à jour les données dans une ou plusieurs tables. La syntaxe est:

```
UPDATE NomTable  
SET NouvelleValeur,...  
WHERE Critères;
```

- Par exemple:

```
UPDATE Livres  
SET Livres.Prix = Livres.Prix + 1.00  
WHERE Livres.Prix < 25.00
```

- Cette requête augmente de 1\$ le prix des livres dont le prix est inférieur à 25\$.



# DELETE

## ● L'instruction DELETE

- DELETE est utilisée pour supprimer des lignes d'une table.

La syntaxe est:

```
DELETE
```

```
FROM NomTable
```

```
WHERE Critère
```

- L'action de l'instruction DELETE est irréversible! Donc faire attention.

- Un exemple:

```
DELETE
```

```
FROM LIVRES
```

```
WHERE PubID="3";
```

# INSERT INTO

## ● L'instruction INSERT INTO

- INSERT INTO est conçue pour insérer de nouvelles lignes dans une table. La syntaxe est:

```
INSERT INTO Cible[(NomChamp,...)]  
VALUES(Valeur1,...)
```

- Où Cible est le nom de la table dans laquelle on veut insérer des données.
- Si on ne spécifie pas de NomChamp alors il faut mettre des valeurs pour tous les champs de la table.
- L'instruction suivante ajoute une ligne à la table LIVRES:  

```
INSERT INTO LIVRES  
VALUES("1-000-00000-0","SQL",1,25.00);
```

# INSERT INTO

- L'instruction suivante ajoute une ligne à la table LIVRES. Toutefois, les colonnes Prix et PubID auront des valeurs NULL:

```
INSERT INTO LIVRES(ISBN,Titre)
VALUES("1-111-11111-1","Partie pêché");
```

- Pour insérer plusieurs lignes, on utilise cette syntaxe:

```
INSERT INTO Cible[(NomChamp,...)]
SELECT NomChamp,...
FROM ExpressionTable
```

- Si Cible est le nom de la table où l'on veut insérer, ExpressionTable est le nom de la table d'où on veut prendre les données pour les insérer dans Cible.

# INSERT INTO

- Supposons que NewLivres est une table de 3 champs: ISBN, PubID, et Prix.
- L'instruction suivante insère des lignes de LIVRES dans NewLivres. Elle insère uniquement les livres dont Prix > 20.00\$.

```
INSERT INTO NewLivres  
SELECT ISBN, PubID, Prix  
FROM Livres  
WHERE Prix > 20;
```

# SELECT INTO

- L'instruction SELECT ... INTO

- L'instruction SELECT... INTO crée une nouvelle table et insère des données d'autres tables. La syntaxe est:

```
SELECT NomChamp,...  
INTO NomNouvelleTable  
FROM Source  
WHERE Condition  
ORDER BY ConditionOrdre
```

- NomChamp est le nom du champ à copier dans la nouvelle table.
- Source est le nom de la table d'où proviennent les données. On peut mettre aussi une requête ou une jointure.

# SELECT INTO

- Par exemple, l'instruction suivante crée une nouvelle table appelée LivresChers ET INCLUT LES LIVRES DE LA TABLE livres qui valent plus de 45.00\$:

```
SELECT Titre, ISBN  
INTO LivresChers  
FROM Livres  
WHERE Prix > 45  
ORDER BY Titre;
```

# SOUS-REQUETES

- Requêtes secondaires
  - SQL permet l'utilisation d'instructions SELECT au sein des instructions suivantes:
    - Autres instructions SELECT
    - Instructions SELECT ... INTO
    - Instructions INSERT...INTO
    - Instructions DELETE
    - Instructions UPDATE
  - L'instruction SELECT interne est qualifiée de requête secondaire et est généralement utilisée dans la clause WHERE de la requête principale ou dans la clause FROM.
  - Dans la clause FROM, elle remplace la table concernée.

# SOUS-REQUETES

- Par exemple, voici une requête de sélection simple:

```
SELECT * FROM Livres
```

```
WHERE prix < 25
```

- On peut ici remplacer le table Livres par une sous-requête

```
SELECT * FROM (SELECT livres.* FROM Livres, Editeurs  
    WHERE Livres.PubID=Editeurs.PubId AND PubNom="alpha")
```

```
WHERE prix < 25
```

- Il s'agit ici d'une projection d'une projection.
- La sous-requête est d'abord effectuée et on y trouve les livres dont le nom de l'éditeur est "alpha"
- Ensuite, de cette sous-requête est extrait les livres dont le prix est inférieur à 25.



# SOUS-REQUETES

- Dans une clause WHERE, la syntaxe d'une requête secondaire prend trois formes possibles:
- Syntaxe 1
  - Comparaison (RequeteSQL)
    - où Comparaison est une expression suivie par une relation de comparaison qui compare l'expression avec la valeur retournée par la requête secondaire.
- Par exemple, l'instruction suivante retourne tous les titres et prix de livres de la table LIVRES, dont les prix sont supérieurs aux prix maximum des livres de la table LIVRES2:

```
SELECT Titre, Prix FROM Livres  
WHERE Prix > (SELECT Max(Prix) FROM Livres2)
```

# SOUS-REQUETES

- Syntaxe 2

Expression [NOT] IN (RequeteSQL)

- Cette syntaxe est utilisée pour consulter une valeur de colonne dans la table résultat d'une autre requête.
- Par exemple, la déclaration suivante retourne tous les titres de la table LIVRES qui n'apparaissent pas dans la table Livres2:

```
SELECT Titre FROM Livres
```

```
WHERE Titre NOT IN (SELECT Titre FROM Livres2);
```

- Syntaxe 3

[NOT] EXISTS (RequeteSQL)

- Cette syntaxe est utilisée pour vérifier si un item existe (si retourné) dans la requête secondaire.

# SOUS-REQUETES

- Par exemple, l'instruction suivante sélectionne tous les éditeurs qui n'ont pas de livres dans la table LIVRES:

```
SELECT PubNom FROM Editeurs WHERE NOT EXISTS  
(SELECT * FROM Livres WHERE Livres.PubID =  
    Editeurs.PubID);
```

- Notes:
  - Avec les syntaxes 1 ou 3, la requête secondaire doit retourner une seule colonne sinon il y a erreur.
  - L'instruction SELECT d'une requête secondaire obéit au même format et aux mêmes règles que toute instruction SELECT. Toutefois, elle doit être placée entre parenthèse.

# Differences entre les SGBDs

|           | Paradox | Access | Sybase | SQL Server | Oracle |
|-----------|---------|--------|--------|------------|--------|
| GROUP BY  | oui     | oui    | oui    | oui        | oui    |
| HAVING    | oui     | oui    | oui    | oui        | oui    |
| UNION     | oui     | oui    | oui    | oui        | oui    |
| INTERSECT | non     | non    | non    | non        | oui    |
| EXCEPT    | non     | non    | non    | non        | MINUS  |

Comment peut-on définir alors l'intersection et la différence si le SGBD ne le supporte pas??

# Syntaxe 2: difference et intersection

- Avec la syntaxe 2, on peut définir l'intersection et la différence (vues en algèbre)
- Intersection  $\text{Table1} \cap \text{table2}$  :
  - **SELECT** attribut1, attribut2, ... **FROM** table1  
**WHERE** attribut1 **IN** (**SELECT** attribut1 **FROM** table2) ;
  - **SELECT** n°enseignant, NomEnseignant **FROM** E1  
**WHERE** n°enseignant **IN** (**SELECT** n°enseignant **FROM** E2) ;

# Syntaxe 2: difference et intersection

## ● Difference Table1-Table2:

- **SELECT** attribut1, attribut2, ... **FROM** table1  
**WHERE** attribut1 **NOT IN** (**SELECT** attribut1 **FROM** table2) ;
- **SELECT** n°enseignant, NomEnseignant **FROM** E1  
**WHERE** n°enseignant **NOT IN** (**SELECT** n°enseignant **FROM** E2) ;

# Les vues

- Les vues permettent d'assurer l'objectif d'**indépendance logique**. Grâce à elles, chaque utilisateur pourra avoir sa vision propre des données.
- Les utilisateurs pourront consulter la base, ou modifier la base (avec certaines restrictions) à travers la vue, c'est-à-dire manipuler la table résultat du SELECT comme si c'était une table réelle.

# Les vues

## ● Syntaxe:

- `CREATE VIEW nom_vue [(nom_col1,...)]  
AS SELECT ... [WITH CHECK OPTION] ;`
- Une fois créée, une vue s'utilise comme une table. Il n'y a pas de duplication des informations mais stockage de la définition de la vue.



# Les vues

- Exemple:

```
CREATE VIEW emp10 AS SELECT * FROM  
emp WHERE n_dept = 10 ;
```

- si la vue emp10 a été créée avec CHECK OPTION on ne pourra à travers cette vue ni modifier, ni insérer des employés ne faisant pas partie du département 10

# Les vues

- Il est possible d'effectuer des INSERT et des UPDATE à travers des vues, sous deux conditions :
  - le SELECT définissant la vue ne doit pas comporter de jointure,
  - les colonnes résultats du SELECT doivent être des colonnes réelles et non pas des expressions.
- **Exemple** : Modification des salaires du département 10 à travers la vue emp10.  
`UPDATE emp10 SET sal = sal *1.1;`

